แบบแผนการพัฒนาโครงงานย่อย (Mini-Project Development Blueprint)

โครงการ: ระบบปรับปรุงคุณภาพเสียงและลดสัญญาณรบกวนแบบเวลาจริง (FPGA Audio Enhancer & Noise Suppression System)

ชื่อไฟล์โปรแกรมหลัก: `enhanced_audio_gui.py`

รายวิชา: Signals & Systems / Digital Signal Processing Practical Labs (ต่อยอดจาก Task 3)

1. ภาพรวมโครงการและวัตถุประสงค์ (Project Overview & Objectives)

โครงงานย่อยนี้พัฒนาขึ้นตามข้อกำหนดในใบงานหน้า 7 ที่กำหนดให้นำข้อมูลสตรีมมิ่งเสียงจากการสุ่มสัญญาณด้วยฮาร์ดแวร์ใน Task 3 (บอร์ด FPGA DE10-Lite MAX 10 ร่วมกับไมโครคอนโทรลเลอร์ ESP32/ESP32-S3) มาประยุกต์ใช้งานจริงบนโปรแกรม GUI ที่มีความสามารถเหนือกว่าการแสดงผลกราฟคลื่นเสียงหรือ FFT พื้นฐาน

- **วัตถุประสงค์หลัก:**
    - พัฒนาระบบประมวลผลสัญญาณเสียงดิจิทัลแบบเวลาจริง (Real-Time DSP Pipeline) บนภาษา Python ที่รับข้อมูลเสียงความถี่สุ่ม Fs = 16 kHz ขนาดบล็อก N = 1024 ตัวอย่าง มาคัดกรองสัญญาณรบกวนและยกระดับคุณภาพเสียงพูดอย่างต่อเนื่อง
    - ออกแบบโครงสร้างโปรแกรมแบบ Multi-Threading ที่แยกเธรดรับข้อมูลและประมวลผลเสียงออกจากเธรดแสดงผล GUI เพื่อป้องกันปัญหาหน้าต่างโปรแกรมค้างหรือเสียงกระตุก
    - สร้างระบบมอนิเตอร์เสียงออกลำโพง/หูฟังแบบสด (Live Speaker Monitoring) พร้อมสวิตช์สลับฟังเปรียบเทียบก่อนและหลังปรับปรุงสัญญาณ (A/B Bypass Switch) และระบบบันทึกไฟล์เสียงมาตรฐาน 16-bit PCM WAV

2. สถาปัตยกรรมระบบแบบครบวงจร (End-to-End System Architecture)

ระบบแบ่งการทำงานออกเป็น 2 ส่วนหลัก ได้แก่ ภาคฮาร์ดแวร์สุ่มและส่งข้อมูล (Task 3 Hardware Pipeline) และภาคซอฟต์แวร์ประมวลผลและแสดงผลบนคอมพิวเตอร์ (Host PC Python Application)

2.1 ภาคฮาร์ดแวร์ต้นทาง (Hardware Data Acquisition & Streaming)

- **โมดูลรับเสียงต้นทาง:** โมดูลไมโครโฟนแอนะล็อก (เช่น MAX9814 หรือ MAX4466) เชื่อมต่อเข้าที่พิน ARDUINO_ADC (A0–A5) ของบอร์ด DE10-Lite MAX 10 FPGA (หรือใช้โมดูลไมโครโฟนดิจิทัล I2S INMP441 / External SPI ADC MCP3201/MCP3202 ตามทางเลือกของใบงาน)
- **การสุ่มสัญญาณบน FPGA (MAX 10):**
    - ใช้วงจรหารความถี่สัญญาณนาฬิกาหลักของ FPGA (50 MHz) เพื่อสร้างสัญญาณควบคุมการสุ่มสัญญาณที่แม่นยำคงที่ Fs = 16,000 Hz (สุ่มทุกๆ 3,125 ลูกคลื่นนาฬิกา หรือคาบเวลา 62.5 ไมโครวินาที)

    - จัดเก็บข้อมูลที่สุ่มได้ลงในหน่วยความจำ Dual-Port Block RAM ขนาดบล็อกละ N = 1024 ตัวอย่าง (ระยะเวลาต่อบล็อก = 64 มิลลิวินาที หรือประมาณ 15.625 บล็อกต่อวินาที)
- **การส่งผ่านข้อมูลผ่าน ESP32 สู่คอมพิวเตอร์:**
    - บอร์ด ESP32/ESP32-S3 รับบล็อกข้อมูล 1024 ตัวอย่างจาก FPGA ผ่านโปรโตคอล SPI (ESP32 เป็น Master, FPGA เป็น Slave) หรือ Handshake Parallel Interface โดยเชื่อมต่อสายกราวด์ร่วมกัน (Common GND)
    - ESP32 ส่งต่อเฟรมข้อมูลแบบไบนารี (Binary Frame) ผ่านพอร์ต USB-to-Serial ด้วยความเร็วสูง (เช่น Baud Rate 921,600 หรือ 2,000,000 bps)
    - **โครงสร้างเฟรมข้อมูลไบนารี (แนะนำ):** Header Sync 2 ไบต์ (เช่น `0xAA`, `0x55`) ตามด้วย Payload ข้อมูลเสียง 1024 ตัวอย่าง (ชนิดข้อมูล 16-bit Little-Endian จำนวน 2,048 ไบต์)

2.2 สถาปัตยกรรมซอฟต์แวร์ (`enhanced_audio_gui.py`)

- Background Worker Thread (`_worker_read_and_dsp`):
    - อ่านเฟรมไบนารีจากพอร์ต Serial แบบบล็อกต่อบล็อก (1024 ตัวอย่าง)
    - ประมวลผลสัญญาณผ่าน DSP Pipeline ทั้ง 4 ขั้นตอน
    - ส่งข้อมูลเสียงที่ประมวลผลเสร็จแล้วไปยังบัฟเฟอร์เสียงของไลบรารี `sounddevice` เพื่อออกลำโพง/หูฟังทันที
    - บันทึกข้อมูลลงไฟล์ `.wav` เมื่อผู้ใช้อยู่ในโหมดบันทึกเสียง
    - ส่งสำเนาข้อมูลบล็อกดิบ (Raw) และบล็อกที่ปรับปรุงแล้ว (Enhanced) เข้าสู่คิวข้อมูล (Thread-Safe Queue) เพื่อให้ GUI นำไปวาดกราฟ
- Main GUI Thread:
    - แสดงผลกราฟสัญญาณในโดเมนเวลา (Time-Domain) และโดเมนความถี่ (FFT Spectrum / Spectrogram) ด้วยอัตราการรีเฟรชที่ลื่นไหล (เช่น ทุกๆ 30–50 มิลลิวินาที)
    - รับค่าการปรับตั้งจากผู้ใช้ผ่านแถบสไลเดอร์ (Sliders) และปุ่มควบคุม (Buttons) แล้วอัปเดตตัวแปรพารามิเตอร์ให้ Worker Thread ใช้งานแบบเวลาจริง

3. รายละเอียดสายพานการประมวลผลสัญญาณดิจิทัล (Real-Time DSP Pipeline Specification)

ข้อมูลแต่ละบล็อก `x_raw[n]` จำนวน N = 1024 ตัวอย่าง จะถูกประมวลผลตามลำดับขั้นดังต่อไปนี้:

ขั้นที่ 1: การลบระดับไฟตรงและการปรับสเกล (DC Offset Removal & Scaling)

- แปลงข้อมูลไบนารี 1024 ตัวอย่างเป็นอาร์เรย์ NumPy ชนิดทศนิยม (`np.float64`)
- ลบค่าเฉลี่ยไฟตรง (DC Offset) ออกจากบล็อกข้อมูลเพื่อให้สัญญาณแกว่งตัวสมมาตรรอบแกนศูนย์:
    - `x_zero = x_raw - np.mean(x_raw)`
- ปรับสเกลแอมพลิจูดให้อยู่ในพิกัดมาตรฐาน 16-bit Signed PCM (ช่วง -32,768 ถึง +32,767) เช่น หากข้อมูลต้นทางมาจาก ADC 12-bit (0 ถึง 4095) ให้คูณด้วยตัวคูณสเกล (เช่น `× 16.0`) เพื่อให้ระดับสัญญาณเหมาะสมกับการประมวลผลและส่งออกลำโพง

ขั้นที่ 2: ตัวกรองความถี่แถบผ่าน IIR Butterworth อันดับที่ 4 (4th-Order IIR Butterworth Bandpass Filter)

- **การออกแบบตัวกรอง:** ใช้ฟังก์ชัน `scipy.signal.butter` คำนวณสัมประสิทธิ์ในรูปแบบ Second-Order Sections (SOS) เพื่อป้องกันปัญหาความไม่เสถียรทางตัวเลข (Numerical Instability):
    - `sos = scipy.signal.butter(N=4, Wn=[f_low, f_high], btype='bandpass', fs=16000, output='sos')`
- **ช่วงความถี่ตัดที่ปรับได้แบบเวลาจริง:**
    - **ความถี่ตัดช่วงต่ำ (High-Pass Cutoff, `f_low`):** ช่วง 80 – 500 Hz (ค่าเริ่มต้นแนะนำ: 120 Hz) ทำหน้าที่ตัดสัญญาณรบกวนความถี่ต่ำ เช่น เสียงฮัมไฟบ้าน 50/60 Hz เสียงสั่นสะเทือนของโต๊ะ และเสียงลมกระแทกไมโครโฟน
    - **ความถี่ตัดช่วงสูง (Low-Pass Cutoff, `f_high`):** ช่วง 1,000 – 7,500 Hz (ค่าเริ่มต้นแนะนำ: 5,000 Hz) ทำหน้าที่ตัดเสียงซ่าความถี่สูง (High-Frequency Hiss / Thermal Noise) และสัญญาณรบกวนใกล้ความถี่ Nyquist (8,000 Hz)
- **การรักษาความต่อเนื่องระหว่างบล็อก (Stateful Filtering):**
    - ต้องใช้ฟังก์ชัน `y_filt, zi = scipy.signal.sosfilt(sos, x_zero, zi=zi)`
    - ตัวแปรสถานะ `zi` (Initial/Final Filter Delays) ของบล็อกปัจจุบันต้องถูกเก็บไว้และส่งต่อไปยังบล็อกถัดไปเสมอ เพื่อป้องกันไม่ให้เกิดเสียงคลิกหรือรอยต่อสัญญาณกระโดดที่ขอบบล็อกทุกๆ 64 มิลลิวินาที

ขั้นที่ 3: ระบบเกตตัดเสียงรบกวนแบบไดนามิก (Dynamic RMS Noise Gate - Soft-Knee Attenuation)

- คำนวณค่าพลังงานเฉลี่ยรากที่สองของกำลังสอง (RMS) ของบล็อกสัญญาณที่ผ่านตัวกรองแล้ว:
    - `rms = np.sqrt(np.mean(y_filt ** 2))`
- เปรียบเทียบค่า `rms` กับเกณฑ์ `Threshold` ที่ผู้ใช้ตั้งไว้ผ่านสไลเดอร์:
    - หาก `rms >= Threshold` หรือ `Threshold == 0`: กำหนดอัตราขยาย `attenuation = 1.0` (ปล่อยเสียงพูดผ่านตามปกติ)
    - หาก `rms < Threshold`: ลดทอนสัญญาณลงอย่างนุ่มนวลด้วยสมการกำลังสอง (Soft-Knee Quadratic Curve):
        - `attenuation = (rms / Threshold) ** 2`
- นำค่าลดทอนไปคูณกับบล็อกสัญญาณ:
    - `y_gated = y_filt * attenuation`
    - ผลลัพธ์ที่ได้จะช่วยกดพื้นหลังในสภาวะเงียบให้เรียบสนิทโดยไม่เกิดอาการเสียงตัดขาดห้วนหรือเสียงคลิกเมื่อเริ่มออกเสียงพูด

ขั้นที่ 4: การขยายระดับเสียงและการจำกัดแอมพลิจูด (Enhance Gain Boost & Soft Limiter)

- ขยายแอมพลิจูดของเสียงพูดที่ผ่านการคัดกรองแล้วด้วยตัวคูณ `Gain_Boost` (ปรับค่าได้ตั้งแต่ 1.0× ถึง 5.0×):
    - `y_boost = y_gated * Gain_Boost`
- จำกัดเพดานแอมพลิจูดสูงสุดและต่ำสุด (Hard/Soft Limiter) ไม่ให้เกินพิกัดของระบบ 16-bit Signed Integer เพื่อป้องกันอาการ Integer Overflow และเสียงแตกพร่า (Clipping):
    - `y_enhanced = np.clip(y_boost, -32767, 32767).astype(np.int16)`

4. ระบบมอนิเตอร์เสียงสดและการบันทึกไฟล์ (Live Monitoring, A/B Bypass & WAV Recorder)

ฟีเจอร์การทำงาน	รายละเอียดการออกแบบทางเทคนิค
Live Speaker Monitoring	ใช้ไลบรารี `sounddevice.OutputStream(samplerate=16000, blocksize=1024, channels=1, dtype='int16')` ส่งบล็อกเสียงออกสู่ลำโพงหรือหูฟังของคอมพิวเตอร์แบบเวลาจริงโดยมีความหน่วงต่ำ
A/B Bypass Switch (Hear Raw)	มีปุ่มสลับโหมดการฟังแบบทันที: โหมดปกติจะส่ง `y_enhanced` ออกลำโพง ส่วนโหมด Bypass (Hear Raw) จะส่ง `x_zero` (สัญญาณดิบที่ลบเพียง DC Offset และจำกัดขนาด) ออกลำโพง เพื่อเปรียบเทียบคุณภาพเสียงก่อนและหลังผ่าน DSP ได้ทันที
16-bit PCM WAV Recorder	ใช้ไลบรารีมาตรฐาน `wave` ของ Python สร้างไฟล์ `.wav` ตั้งค่า Mono (1 Channel), Sample Width = 2 Bytes (16-bit), Frame Rate = 16,000 Hz บันทึกข้อมูล `y_enhanced.tobytes()` ต่อเนื่องทุกบล็อกเมื่อเปิดการบันทึก และตั้งชื่อไฟล์ตามวันเวลาอัตโนมัติ (เช่น `enhanced_record_YYYYMMDD_HHMMSS.wav`)

5. การออกแบบส่วนติดต่อผู้ใช้ (GUI Layout & Interactive Controls)

หน้าต่างโปรแกรมออกแบบให้แบ่งเป็น 2 โซนหลัก ได้แก่ โซนแสดงผลกราฟวิเคราะห์สัญญาณ และโซนแผงควบคุมพารามิเตอร์ DSP

5.1 โซนแสดงผลกราฟแบบเวลาจริง (Real-Time Visualization Panel)

- **กราฟโดเมนเวลา (Time-Domain Waveform Plot):**
  - แสดงคลื่นเสียงขนาด 1024 ตัวอย่าง (แกนเวลา 0 ถึง 64 มิลลิวินาที)
  - พล็อตเปรียบเทียบ 2 เส้นซ้อนกัน: **Raw Signal** (เส้นสีเทา/โปร่งแสง) และ **Enhanced Signal** (เส้นสีส้มหรือสีฟ้าสว่าง)
  - แสดงเส้นประแนวนอนระบุระดับ $\pm$`Threshold` ของ Noise Gate เพื่อให้ผู้ใช้เห็นเกณฑ์การตัดเสียงอย่างชัดเจน
- **กราฟสเปกตรัมความถี่ (Frequency-Domain FFT Spectrum Plot):**
  - คำนวณ FFT ขนาด N = 1024 จุด (ความละเอียดความถี่ Δf = 16,000 / 1024 = 15.625 Hz) โดยคูณด้วยหน้าต่าง **Hann Window** (`np.hanning(1024)`) ก่อนทำ FFT เพื่อลด Spectral Leakage ตามหลักการในใบงาน
  - แสดงขนาดสเปกตรัมในหน่วยเดซิเบล (dB) บนแกนความถี่ 0 ถึง 8,000 Hz เปรียบเทียบระหว่าง Raw Spectrum กับ Enhanced Spectrum

- แสดงเส้นประแนวตั้งระบุตำแหน่งความถี่ตัด `f_low` และ `f_high` ที่กำลังตั้งค่าอยู่

5.2 โซนแผงควบคุมและแถบสถานะ (Interactive Control Panel & Status Bar)

- **แถบสไลเดอร์ปรับค่าแบบเวลาจริง (4 Sliders):**
  - `High-Pass Cutoff (Hz)`: ช่วง 80 – 500 Hz (ความละเอียดขั้นละ 5 หรือ 10 Hz)
  - `Low-Pass Cutoff (Hz)`: ช่วง 1,000 – 7,500 Hz (ความละเอียดขั้นละ 50 หรือ 100 Hz)
  - `Noise Gate Threshold`: ช่วง 0 – 5,000 (ระดับแอมพลิจูด RMS ในสเกล 16-bit)
  - `Enhance Gain Boost`: ช่วง 1.0× – 5.0× (ความละเอียดขั้นละ 0.1×)
- **ปุ่มควบคุมการทำงาน (Action Buttons):**
  - ปุ่มสลับ `Bypass Filter (Hear Raw: ON / OFF)`
  - ปุ่ม `Start / Stop Recording (.wav)`
  - ปุ่มสลับโหมดแหล่งข้อมูล `Source: Serial Hardware / Demo Simulation` (ช่วยให้ทดสอบและสาธิตระบบได้แม้ไม่ได้ต่อบอร์ดจริง)
- **แถบแสดงสถานะและมาตรวัด (Live Telemetry Indicators):**
  - แสดงค่า `Current RMS Level` เทียบกับ `Threshold`
  - แสดงค่า `Noise Gate Attenuation (%)` หรือระดับการกดสัญญาณในขณะนั้น
  - แจ้งเตือน `CLIPPING WARNING` เมื่อแอมพลิจูดแตะระดับ ±32,767

6. โครงสร้างโมดูลและคลาสของโค้ด (`enhanced_audio_gui.py` Code Blueprint)

- **ไลบรารีที่ต้องใช้งาน (Required Dependencies):**
  - `numpy` (จัดการอาร์เรย์ คำนวณ RMS, Hann Window, FFT และ Clipping)
  - `scipy.signal` (`butter`, `sosfilt`, `sosfilt_zi` สำหรับตัวกรอง IIR Butterworth)
  - `pyserial` (`serial` สำหรับรับสตรีมข้อมูลไบนารีจาก ESP32)
  - `sounddevice` (สำหรับส่งเสียงออกลำโพงแบบเวลาจริง)
  - `wave` (ไลบรารีมาตรฐานสำหรับบันทึกไฟล์ `.wav`)
  - `tkinter` ร่วมกับ `matplotlib.backends.backend_tkagg` (หรือ `PyQt5` / `pyqtgraph` สำหรับสร้างหน้าต่าง GUI และกราฟความเร็วสูง)
- **โครงสร้างเมธอดหลักภายในคลาส `AudioEnhancerGUI`:**
  - `__init__(self, root)`: กำหนดค่าคงที่ (`FS = 16000`, `BLOCK_SIZE = 1024`), สร้างตัวแปรพารามิเตอร์ DSP, เรียกสร้างส่วนประกอบ GUI และเริ่มต้นระบบเสียง
  - `_design_bandpass_sos(self, lowcut, highcut)`: คำนวณสัมประสิทธิ์ `sos` อันดับ 4 พร้อมตรวจสอบเงื่อนไขความปลอดภัย (`lowcut < highcut`) และรีเซ็ตหรือปรับสถานะ `zi`

- `_sync_and_read_block(self, ser)`: ตรวจจับ Header Bytes (`0xAA 0x55`) จากพอร์ต Serial และอ่านไบต์ข้อมูลครบ 2,048 ไบต์ แปลงเป็น `np.frombuffer(..., dtype=np.int16)` (หรือจำลองสัญญาณในโหมด Simulation)
- `_process_dsp_block(self, raw_block)`: รันสายพาน DSP ครบ 4 ขั้นตอน (DC Removal -> SOS Bandpass -> Soft-Knee RMS Gate -> Gain Boost & Clip) คืนค่าทั้ง `raw_zero` และ `enhanced_block` พร้อมค่า `rms` และ `attenuation`
- `_worker_read_and_dsp(self)`: ลูปการทำงานใน Background Thread ที่เชื่อมโยงการอ่านข้อมูล การประมวลผล DSP การส่งเสียงเข้า `sounddevice` การเขียนไฟล์ `wave` และการส่งข้อมูลเข้า `plot_queue`
- `_update_gui_loop(self)`: ฟังก์ชันที่ถูกเรียกซ้ำด้วย `root.after(30, ...)` เพื่อดึงข้อมูลล่าสุดจาก `plot_queue` มาคำนวณ FFT และอัปเดตเส้นกราฟทั้งสองแกนอย่างลื่นไหล
- `on_close(self)`: หยุดการทำงานของ Worker Thread ปิดพอร์ต Serial ปิดสตรีม `sounddevice` และบันทึกไฟล์ `.wav` ที่ค้างอยู่ให้เรียบร้อยก่อนปิดหน้าต่าง

7. แผนการดำเนินงานและการทดสอบระบบ (Implementation & Testing Roadmap)

ระยะที่ 1: การพัฒนาและทดสอบด้วยสัญญาณจำลอง (Offline Simulation & DSP Verification)

- สร้างฟังก์ชันสร้างสัญญาณทดสอบภายในโค้ด (ประกอบด้วยความถี่เสียงพูดจำลอง 300 Hz + 1,000 Hz ผสมกับเสียงฮัม 50 Hz ความถี่ต่ำ และเสียงซ่าความถี่สูง 6,500 Hz รวมถึงช่วงเวลาเงียบสลับช่วงพูด)
- ตรวจสอบการทำงานของตัวกรอง Butterworth Bandpass ว่าสามารถกดความถี่ 50 Hz และ 6,500 Hz ลงได้ตามที่ปรับสไลเดอร์ และตรวจสอบว่าไม่มีเสียงคลิกระหว่างรอยต่อบล็อก
- ทดสอบสมการ Soft-Knee RMS Noise Gate ว่าลดระดับพื้นหลังช่วงเงียบลงได้อย่างนุ่มนวล

ระยะที่ 2: การเชื่อมต่อฮาร์ดแวร์ Task 3 (Hardware-in-the-Loop Integration)

- อัปโหลดโค้ด VHDL/Verilog ลงบอร์ด DE10-Lite MAX 10 FPGA และโค้ด Arduino ลงบอร์ด ESP32/ESP32-S3
- ตรวจสอบอัตราการสุ่มสัญญาณจริงที่ Fs = 16 kHz ด้วยการป้อนสัญญาณไซน์ความถี่มาตรฐาน (เช่น 1,000 Hz) จากเครื่องกำเนิดสัญญาณ (Function Generator) แล้วดูยอดกราฟ FFT บน GUI ว่าตรงกับ 1,000 Hz พอดี

ระยะที่ 3: การทดสอบใช้งานจริงและเก็บผลการทดลอง (Experimental Validation & Report Collection)

ทำการทดสอบและบันทึกผลเปรียบเทียบ (ทั้งภาพแคปเจอร์หน้าจอ GUI และไฟล์เสียง `.wav`) ใน 4 สถานการณ์เพื่อนำไปใส่ในรายงาน:

- **สภาวะเงียบ (Silence / Ambient Noise Test):** เปรียบเทียบระดับพื้นหลังของสัญญาณ (Noise Floor) ก่อนและหลังเปิดใช้งาน Dynamic RMS Noise Gate
- **การทดสอบเสียงพูดจริง (Live Speech Enhancement Test):** ทดสอบพูดผ่านไมโครโฟน ปรับตั้งค่า Bandpass (เช่น 120 Hz – 4,500 Hz) ร่วมกับ Gain Boost สลับโหมด Bypass เพื่อยืนยันความชัดเจนของเสียงพูด

- **การทดสอบตัดสัญญาณรบกวนความถี่จำเพาะ (Targeted Interference Rejection):** เปิดเสียงรบกวนความถี่ต่ำ (เช่น พัดลมหรือโทน 100 Hz) หรือความถี่สูง (เช่น โทน 6,000 Hz) เข้าไมโครโฟน แล้วเลื่อนสไลเดอร์ตัดความถี่ดังกล่าวออกในขณะดูกราฟ FFT แบบสด
- **การทดสอบเพดานแอมพลิจูด (Loud Clap / Soft Limiter Test):** ทดสอบปรบมือหรือส่งเสียงดังใกล้ไมโครโฟน พร้อมปรับ Gain Boost สูง เพื่อยืนยันว่าระบบ `np.clip` ป้องกันปัญหาข้อมูลล้น (Overflow) ได้อย่างถูกต้อง